
PokerBot

George White

Ata Guvendi

Humza Khan

Keluobi Wang

Abstract

We present a poker-playing agent that combines precise mathematical odds calculation with behavioral exploitation, namely the ability to identify and capitalize on patterns and tendencies in opponent play. The agent is built on a tiered memory architecture designed to operate within context constraints, enabling structured reasoning for the underlying language model. We demonstrate that this approach consistently outperforms naive baseline agents and remains competitive against mathematically-driven opponents, with profitability results summarized across agent configurations, including those running under low-resource conditions.

1 Introduction

1.1 Background and Motivation

Games like poker are inherently stochastic and have imperfect information, which is a massive challenge for AI. Unlike games such as chess, where the entire board is visible, the agent is forced to act under uncertainty. Game Theory Optimal (GTO) is currently dominating the poker landscape, by taking a mathematically unexploitable approach to a Nash Equilibrium. However, a Nash Equilibrium assumes opponents are playing perfectly, which is rarely the case. Large Language Models excel at reasoning, pattern recognition, and adapting to novel situations. Theoretically, an LLM could observe a passive player, recognize the pattern, and formulate an exploitative strategy that a math bot couldn't replicate.

1.2 Problem Statement

Math-only bots, which are currently dominating the poker landscape, cannot exploit human irrationalities. For example, if a player decides to play extremely passively for an entire game, then they suffer the risk that other players know when they have a strong hand (when they bet big). Agents who can correctly identify these player types, reason about how to exploit them, and update the mathematically optimal play should be a much superior player compared to bots without any reasoning.

However, realizing this capability through modern LLMs presents its own structural challenges. While LLMs excel at reasoning, base models lack the structured long-term memory and integrated probabilistic tools required to consistently navigate a complex, uncertain environment without degrading their context windows.

1.3 Proposed Approach

Our approach to use an LLM Core as our agents brain, giving it access to both an equity calculator and memory modules. The equity calculators serves as a more precise metric for hand strength that the agent can call on whenever it is unsure or can't just rely on reasoning. The memory modules serve to store data from hands, games, and players, so that the Agent can learn over time where it can best exploit its opponents. We split the memory into short-term (hand history), medium-term (game history), and long-term (opponent history).

2 Related Work

2.1 Game-Theoretic Poker Agents

Poker has long served as a benchmark for decision making under uncertainty. State-of-the-art poker systems such as Libratus [Brown and Sandholm, 2018] and Pluribus [Brown and Sandholm, 2019] achieved superhuman performance through game-theoretic reasoning and Counterfactual Regret Minimization (CFR) techniques [Zinkevich et al., 2007]. These systems approximate Nash-equilibrium strategies and are highly effective in imperfect-information environments. However, equilibrium-based approaches are designed to be unexploitable rather than adaptive. As a result, they generally do not maintain explicit models of individual opponents and are limited in their ability to exploit persistent behavioral tendencies or non-rational human actions such as emotional betting and bluffing patterns [Billings et al., 2004, Albrecht and Stone, 2018].

2.2 LLM-Based Agents and Tool Use

Recent work has demonstrated that Large Language Models (LLMs) can function as autonomous agents by combining reasoning, planning, and external tool use. ReAct [Yao et al., 2023] introduced an interleaved reasoning-and-acting framework that enables LLMs to query tools and update decisions based on observations. Reflexion [Shinn et al., 2023] further extended this paradigm by allowing agents to learn from previous failures through self-reflection and verbal feedback. These approaches have shown strong performance in interactive environments; however, they typically operate with limited memory structures and do not explicitly model uncertainty or opponent beliefs. Consequently, their applicability to strategic games with hidden information remains limited.

2.3 Memory-Augmented Agents

Several recent studies have explored memory as a mechanism for improving long-term decision making in LLM agents. Generative Agents [Park et al., 2023] introduced a memory stream that stores observations and retrieves relevant experiences for future reasoning. Voyager [Wang et al., 2023] demonstrated the effectiveness of skill libraries and iterative memory accumulation in open-ended tasks. More recently, AnyTool [Du et al., 2024] proposed a hierarchical architecture that combines an LLM controller with structured tool selection and self-reflection mechanisms. While these systems successfully improve planning and task execution, they are not designed for adversarial environments where beliefs about other agents must be continuously updated.

Our work combines insights from these three research directions. We integrate probabilistic poker tools, hierarchical memory, and an LLM-based reasoning core into a unified framework for opponent-aware decision making. Unlike equilibrium-based poker agents, our system explicitly models opponent behavior over time. Compared to existing LLM agents, our approach introduces structured memory and domain-specific reasoning tools designed for stochastic multi-agent environments.

3 Method

3.1 System Overview

The combined system contains four key parts: the poker simulation environment, the LLM decision maker, the memory system for the LLM, and the mathematical equity calculator tool. Figure 1 shows a graphical representation of the system.

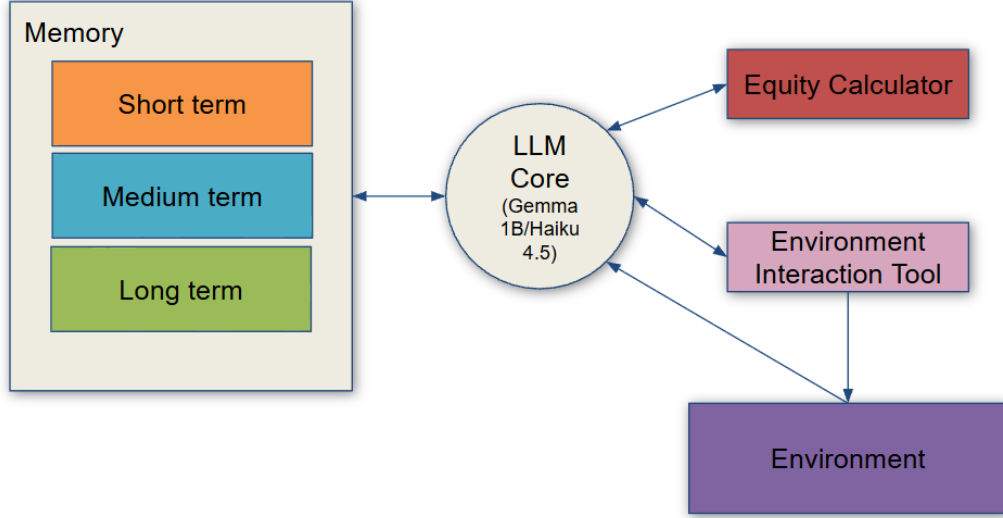


Figure 1: Representation of the full PokerBot system

3.2 Environment and State Tracking

To build and evaluate the agent, we utilized the open-source Python library PokerKit as our primary environment. PokerKit provides three essential computational features that serve as the foundation for our system:

1. **Robust Poker Simulation:** PokerKit strictly enforces the rules of No-Limit Texas Hold'em, ensuring state integrity by preventing the LLM or algorithmic opponents from taking illegal actions. Furthermore, it accurately manages all internal pot calculations, side pots, and final hand evaluations.
2. **Scalability:** Fast simulation speeds are critical for generating sufficient hand histories. PokerKit efficiently simulates hundreds of hands in seconds when standard algorithmic bots are used. This scalability provides a high performance baseline that offsets the computational latency introduced by querying the LLM core.
3. **Monte Carlo Integration:** PokerKit includes a built in simulator capable of executing thousands of randomized board runouts rapidly. This core functionality is directly utilized by our agent's mathematical toolkit to calculate live hand strength.

State Tracking and Extraction: Beyond game mechanics, PokerKit serves as the data pipeline for the LLM. As the game progresses, our system queries the PokerKit state object to extract live variables, such as current stack sizes, pot odds, community cards, and the sequential action history of the current hand. This raw environment data is then formatted into a structured text prompt and injected into the agent's short-term and medium-term memory, effectively translating the mathematical game state into readable context for the LLM.

3.3 Memory Modules

As outlined in Figure 1, the memory system comprises of three individual tiers, meant to reflect the organization of the poker game:

- Short term is responsible for managing the hand-level context of the agent. A "hand" refers to the full run-out and resolution of a single pot.
- Medium term manages the context for the game. The "game" is a series of hands, in which the opponents do not rotate, change chip counts (except for betting outcomes), etc.

- Long term records information over the entire lifecycle of the bot, which retains context about all observed games.

3.3.1 Short Term Memory

We manage the short term information without any tools, injecting content directly into the agent’s context. This helps reduce the number of tools and the decision loops the agent engages in, particularly because the short-term memory has an upper bound in terms of length. We inject the hand’s betting history, turn order and the state of the board to the agent in full inside the system prompt, asking for its action alongside providing available tools.

3.3.2 Medium Term Memory

Medium term memory is organized in terms of numbered games. In particular, for each game with an incrementing identifier of the form 001, 002, etc., we create the following files:

- metadata: JSON file denoting player names, time and number of the game
- rawlog: text file containing all PokerKit outputs
- stats: JSON file containing action counts, and chip changes per player
- trends: text file containing agent’s reasoning and evaluation of the current hand, which it can contribute to retrospectively after each resolution of the pot

The particular file of interest is trends.txt, where the agent can be thought of as keeping a diary of the game. This allows the agent to note weaknesses in play of individual opponents and local trends, which in theory, offers the ability to extend the agent’s capabilities by exploiting the opponents beyond what the traditional odds would allow. Below is an abridged trends file:

```

=== TREND LOG | GAME 007 ===
...
[After hand #2]
P5 demonstrated both aggressive and deceptive play in this hand. Key observations:

AGGRESSIVE PLAY:
- P5 made a well-timed 3-bet to 12,000....

HOLDING AND REVEALED HAND:
- P5 showed 7c-Ts (a pair of sevens), which was a marginal holding...

STRATEGIC ASSESSMENT:
- P5's 3-bet was aggressive but somewhat loose...
- However, P5 got lucky...
- P5 called down ...

WEAKNESSES TO EXPLOIT:
- P5 appears....

```

The key part we call attention to is the “Weaknesses to Exploit” section, in which the agent develops a strategy that it can recall in future time-steps within the game. Given the running nature of this section, this can be thought of as a dynamic scratchpad from which the agent ideates, or one which guides the agent’s CoT in a structured manner.

3.3.3 Long Term Memory

Long term memory manages cross-game context on a player-by-player basis. The main inspiration behind this module is MongoDB’s document like storage, which we adopt in an even more unstructured document type storage. The underlying motivation is that of context management, where considering the unbounded quantity of players and games, it is more apt to organize the memory by a player-by-player basis. Thus, the agent can selectively draw up files on the active opponents (not everyone in the table, but indeed, a subset of the table players currently being dueling with) to keep context utilization low. We organize long term memory in the following manner:

- index: A JSON file keeping short summaries of each player. An example looks like:

```

"P4": {
  "games_observed": 150,
  "last_seen": "2026-05-30T23:39:24.650770",
  "summary": ...,
  "tags": [
    "passive",
    "balanced"
  ]
}

```

where the tags serve to offer very low token (<10) profiles of the opponent of interest, and the summary keeps a comparable entry to the last medium term trends entry so that the agent has a baseline to access against a new opponent in a fresh table.

- A document for each player containing all the information in the index, and an attached observations log that the agent can use tools to append to. An example observation is:

```

[2026-05-30T21:52:13.737830 | 002]
P2 (Big Blind): Posted 2000 as the big blind.
When facing P5's raise to 8000, P2 folded.
This was a passive decision, though reasonable
given the short-handed situation and uncertain
hand strength.

```

- And self, a text file of the same format, concerning the agent's observations about itself retrospectively. We denote this as a "judge", where the LLM call dissociates itself from the agent to analyze its own behavior as if it were an opponent.

3.4 Equity Calculator

The agent can call upon the equity calculator to get a better idea of hand strength. The calculator uses Monte-Carlo simulations, which are built into PokerKit, to evaluate hand strength. The calculator can seamlessly simulate 1000 hands, where the rest of the board cards and the opponents cards are chosen and randomly, and then return the percentage of hands that the agent won.

3.5 LLM Core and Decision Prompting

The actual LLM core consists of a few different parts. First, there is the actual LLM decision maker, which is fed the game state and various pieces of memory (as explained above), and makes decisions about whether to call a given tool or to take a specific action. We use Anthropic's Haiku 4.5 model through the API for this, as it allows us to minimize API costs. The system prompt for this model is as follows:

```

You are a poker agent playing {number of players}-handed Texas Hold'em
→ No-Limit.
Blinds 1000/2000. You are Hero. Think about pot odds, position, hand
→ strength, stack depth.
Use the take_action tool to act.
Use self-notes and opponent notes to inform your decisions.

```

Notably, this is much shorter than our original system prompt, as we continually hit token limits while testing. With this system prompt (modified with memory as above), we feed the short-term memory (i.e., the game state variables like current hole cards and board cards) and ask for an action. One key problem with LLM outputs is that they often do not follow the instructions, and will return an invalid output type or with extraneous information. However, the Anthropic Python SDK has a feature that requires the LLM to return its output via a specific tool that enforces output types. We use this tool to enforce a three-part response: the action, as a string from one of fold, check, call, or raise; the amount, as a float if the LLM decides to raise; and the reasoning, a short three-sentence explanation for the action. Our instructions are passed as a docstring within the function:

Submit your poker action for this turn and give a maximum 3 sentence
→ explanation for why.

Args:

action: One of fold, check, call, raise
amount: Raise amount in chips. Required if action is raise.
reasoning: Reasoning for why you took a poker action. Maximum 3
→ sentences. Rate how aggressive each opponent's actions are on a
→ 1-10 scale, but only if you made it to the flop or later.

Another key part of the LLM core is the judge agent. This agent also uses Haiku 4.5. The goal of the judge agent is to, as the name suggests, evaluate and give feedback on both the agent's actions and the other player's actions. We prompt the judge as follows:

You are an evaluating agent for players playing Texas Hold-em. Your job
→ is to evaluate and provide useful information about the other
→ non-agent players to aid Player 1 in defeating them. Do not
→ evaluate Player 1 (Hero), focus on the other players, denoted P2
→ and up. Remember that index 0 is the agent, index 1 is Player 2 and
→ so on. Use the generate_observation tool to submit your decision.

We get a free-form analysis in return that the memory module can process.

4 Experiments

4.1 Experimental Setup

We evaluated the agent in a real poker game setting against a series of predetermined personalities and strategies. There are six strategies:

1. **Pocket Pair:** This player will only play strong pocket pairs, that is, if the player is dealt a pair of 7s or higher on the initial distribution of cards. They will raise aggressively if they have such a hand.
2. **Punter:** This player will randomly decide (with a 10% chance) if they are punting, meaning if they will aggressively raise the pot. If they do not punt, they will fold.
3. **Tight Passive:** This player will fold with a 90% chance, and check or call otherwise.
4. **Random Raiser:** This player will randomly raise with a 10% chance, fold with a 10% chance, and check or call otherwise.
5. **Expected Value:** This player will employ a mathematical strategy using the equity calculator described earlier. Specifically, if the expected chance of winning is higher than 50%, this player will raise. If the chance is lower than 50%, but higher than the pot odds, the player will check or call. Otherwise, the player will fold.
6. **Expected Value-Conservative:** This player will employ the same strategy as the previous player, but the raise threshold is set to 70%.

Each strategy represents one player. Otherwise, the setup is standard No-Limit Texas Hold-em, with rotating blinds, small blinds of 1,000, big blinds of 2,000, an initial stack of 20,000 for each player, and no antes. We run 10 games, capped at either 30 hands or when the agent goes bust (hits zero chips).

4.2 Main Experiment

Table 3 shows the results of our main specification experiment. We see that the agent performs poorly compared to the conservative EV strategy, but manages to win 30% of games and outperforms the standard EV strategy.

Round	Agent	Pocket Pair	Punter	Passive	Random	EV 0.5	EV 0.7	Agent Win %
1	78,000	33,000	0	19,000	0	10,000	0	24.14
2	0	22,000	58,000	20,000	0	20,000	20,000	0.00
3	8,333	13,333	61,000	9,000	0	48,334	0	20.69
4	0	29,500	1,000	14,000	37,500	0	58,000	10.53
5	71,000	0	61,000	8,000	0	0	0	34.48
6	0	14,000	9,000	17,000	46,000	20,000	34,000	0.00
7	0	0	26,000	13,000	0	58,000	43,000	10.00
8	0	20,000	8,000	20,000	0	43,000	49,000	0.00
9	0	0	7,000	14,000	58,000	0	61,000	0.00
10	53,500	0	0	3,500	51,040	0	31,960	20.69
Mean	19,167	19,238	21,000	13,955	20,102	18,815	27,724	10.96

Table 1: End-of-simulation chip stacks by agent type, with Agent win rate across hands. Bold values indicate the largest chip stack in each row; the final row reports column means.

While poker is a naturally stochastic game, and we would expect some variation, these results indicate that the agent is too aggressive in the aggregate, as it’s able to win a fair share of games (indeed, it is tied for first on this measure), but does so at the expense of extreme risk, as seen by the six times it goes bust, while the EV strategies can win without taking on excessive risk.

This is a particularly interesting result, as the literature (e.g., Jia et al. [2024]) has found strong evidence that LLMs exhibit risk *aversion*, meaning that we would expect to see an LLM here win few games, but rarely go bust.

To an example of this, let’s examine Game 3 (shown in Figure 2, where the agent went bust).

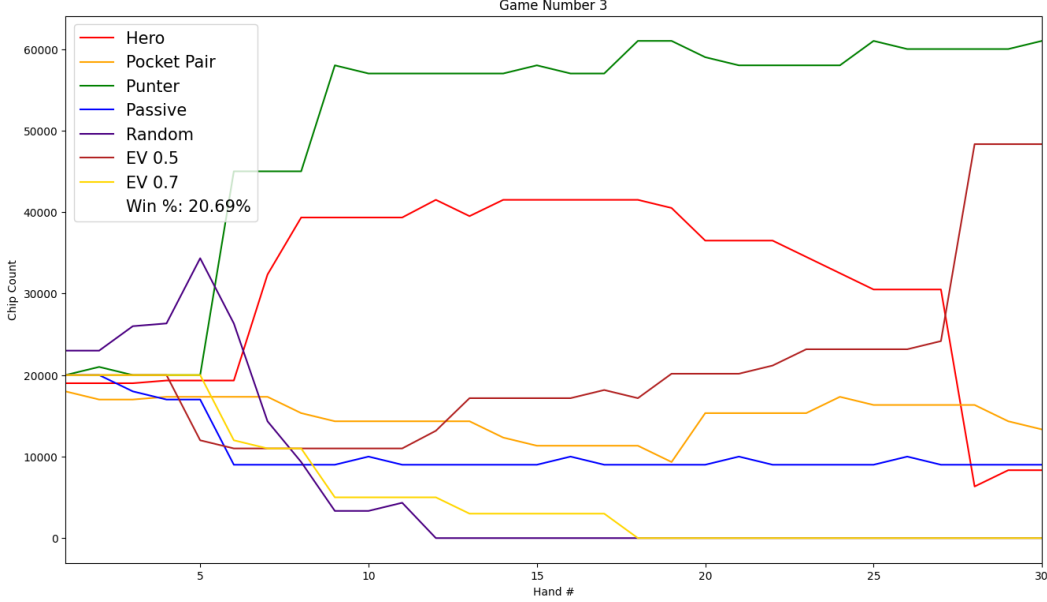


Figure 2: Stack count per player during the course of Game 3. Win % is the percent of hands the Agent (Hero) wins.

We see that the agent performs quite well the whole game, until the very last few hands when it proceeds to lose almost all of its money in a single hand. Putting that much in is an extremely risky and aggressive move.

This result is not the only example - in 7/10 games played, the agent loses a significant amount of money in the last few hands. Of course, since we end the game when the agent goes bust, we might expect to see this. However, given that this happens so frequently, it indicates that the agent has trouble managing risk and relies on highly risky plays to win.

4.3 Ablations

Another theory is that the equity calculator is leading the LLM astray. This comes from the results of our midterm experiment, which tested the LLM without the equity calculator against only the non-EV strategies. All players only have 10,000 chips to start, however. Table 2 shows the results.

	Agent	Pocket Pair	Punter	Passive	Random	Agent Win %
Game 1	33,000	0	43,000	24,000	0	31.03
Game 2	72,000	0	1,000	27,000	0	34.48
Game 3	34,000	22,000	28,000	16,000	0	37.93
Game 4	73,000	0	0	27,000	0	44.83
Game 5	73,000	0	9,000	18,000	0	51.72
Game 6	0	12,500	18,000	18,000	51,500	0
Mean	47,500	5,750	16,500	21,666	8,583	33.33

Table 2: End-of-ablation-simulation chip stacks by agent type, with Agent win rate across hands. Bold values indicate the largest chip stack in each row; the final row reports column means.

The agent performs dramatically better here, indicating that the equity calculator could plausibly be hurting the agent’s performance. One possible reason is that poker hands suffer from selection bias, whereby if a player is still in the game, they are likely to have a better hand than one might get from random chance. Thus, a Monte-Carlo-based method like the equity calculator will *overestimate* the odds of winning, which could lead the agent astray.

We still also see the risky behavior from earlier present, just primarily in the opposite direction. The agent in the first five games wins chiefly due to a few hands that result in a massive windfall for the agent. While this alone is not evidence of risk-taking, in Game 6, the bot loses half its stack in hand 2 and loses the rest a few hands later, indicating a high level of risk when betting.

One final theory as to the performance of the bot is that perhaps the EV-based strategies are particularly strong. Table 3 shows the results of the agent without an equity calculator against the full panel of agents.

Round	Agent	Pocket Pair	Punter	Passive	Random	EV 0.5	EV 0.7	Agent Win %
1	16,000	25,000	36,500	16,500	0	46,000	0	20.69
2	0	37,000	25,000	17,000	0	38,014	22,986	9.09
3	1,500	81,000	0	2,000	0	0	55,500	24.14
4	0	17,000	17,000	17,000	49,000	20,000	20,000	0.00
5	0	13,000	2,000	0	0	80,800	44,200	19.05
6	35,656	7,000	53,000	6,000	0	38,344	0	24.14
7	100,668	19,000	0	14,000	0	6,332	0	31.03
8	0	31,000	0	10,000	0	0	99,000	24.14
9	26,196	0	28,000	14,000	0	0	71,804	31.03
10	107,000	19,000	14,000	0	0	0	0	31.03
Mean	28,702	24,900	17,550	9,650	4,900	22,949	31,349	21.43

Table 3: End-of-second-ablation-simulation chip stacks by agent type, with Agent win rate across hands. Bold values indicate the largest chip stack in each row; the final row reports column means.

We see that the agent performs remarkably better here, while the EV strategies perform around the same. While we see a similar pattern of risk-taking here, with the agent taking many risky gambles, the agent does a better job of holding onto its money. This can be seen in terms of average game

length, with the average game lasting 19.9 hands with the equity calculator and 24.8 hands without. This evidence, combined with the fact that the EV strategies perform roughly the same, provides suggestive evidence that the equity calculator is making the agent meaningfully overconfident and leading it to take many more risky bets that end up not turning out very well.

5 Conclusions

In this project, we set out to build an LLM-based poker agent that combined a tiered memory architecture with an equity calculator, using Haiku 4.5 as its reasoning core. We observed that the agent remained modestly competitive in empirical trials. Our ablations uncovered that the equity calculator may be introducing overconfidence due to selection bias, which also may be leading to worse outcomes. The tiered memory and judge system enabled our model to engage in behavioral modeling and exploitation, a core differentiator from other purely mathematically motivated models. Further, the model’s modular architecture enables future developments that could further expand the capabilities of our agent.

In presenting our conclusions, we note the primary constraint of the empirical experimentation: token cost. Despite the underlying tools providing summaries instead of full pictures, and a non-state-of-the-art LLM, our model performs competitively, especially when considering the long-established reality that long-term poker winnings follow heavy-tailed distributions due to swan events. In the context of these findings, we have a positive outlook on the upper bound of the model’s capabilities when both the LLM and the modules are upgraded to handle more information more accurately.

Similarly, we acknowledge a major limitation of our model, where the token consumption is truly prohibitive to run large-scale experimentation, even under our specification, and will certainly become a consideration to a hypothetical deployer of our agent, especially if guided by profitability concerns.

6 Team Contributions

Github Repository: Visit Repository

G. White: Equity calculator, PokerKit initialization, experiments

H. Khan: LLM core, memory integration, experiments

A. Guvendi: Memory modules, memory integration, experiments

K. Wang: GUI interface and visualization

References

- Stefano V. Albrecht and Peter Stone. Autonomous agents modelling other agents: A comprehensive survey and open problems. *Artificial Intelligence*, 258:66–95, 2018.
- Darse Billings, Aaron Davidson, Jonathan Schaeffer, and Duane Szafron. Approaches to opponent modeling in poker. In *Canadian Conference on Artificial Intelligence*, 2004.
- Noam Brown and Tuomas Sandholm. Superhuman ai for heads-up no-limit poker: Libratus beats top professionals. *Science*, 359(6374):418–424, 2018.
- Noam Brown and Tuomas Sandholm. Superhuman ai for multiplayer poker. *Science*, 365(6456): 885–890, 2019.
- Yu Du et al. Anytool: Self-reflective, hierarchical agents for large-scale api calls. *arXiv preprint arXiv:2402.04253*, 2024.
- Jingru Jia, Zehua Yuan, Junhao Pan, Paul E. McNamara, and Deming Chen. Decision-making behavior evaluation framework for llms under uncertain context, 2024. URL <https://arxiv.org/abs/2406.05972>.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.

- Noah Shinn, Federico Cassano, Bailin Labash, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems*, volume 20, 2007.